

Monitorización en tiempo real del tráfico de datos por usuario (UE) en una estación base 5G srsRAN,

Jin Yunle — Asunción Bueneke — Abel González Gil

Abril 2026

1 Introducción

La monitorización del tráfico de datos por usuario (UE) en redes 5G es esencial para garantizar la calidad de servicio y detectar posibles anomalías en la transmisión. En esta práctica se ha desarrollado una herramienta de monitorización en tiempo real para analizar el tráfico generado por diferentes UEs dentro de una red 5G basada en srsRAN y Open5GS. El objetivo principal consiste en leer un fichero de logs generado por la CU, identificar los mensajes que contienen información sobre el volumen de datos transmitido, agregar dichos datos por usuario y representarlos gráficamente mediante un panel interactivo desarrollado con Dash.

La práctica se apoya en dos scripts principales. El primero, `log-sim.py`, actúa como simulador de logs y genera progresivamente un fichero de salida llamado `cu-lan-ho-live.log` a partir de un log real previamente capturado, `cu-lan-ho.log`. El segundo script, `log-parser.py`, desarrollado para esta práctica, lee ese fichero en tiempo real, procesa las líneas nuevas, extrae los identificadores de UE y los tamaños de PDU, y actualiza una gráfica en Dash.

Esta arquitectura permite reproducir el comportamiento de una red en funcionamiento, donde los logs se generan de forma continua y deben ser procesados sin detener la ejecución del sistema.

2 Contexto técnico de la práctica

En una arquitectura 5G, el tráfico de usuario atraviesa diferentes capas y componentes, como la UE, el gNB, la CU, la DU y el Core 5G. En esta práctica, el análisis se centra en los logs generados por la CU de srsRAN, especialmente en aquellos mensajes relacionados con la capa SDAP.

La capa SDAP se encarga de asociar los flujos de datos del usuario con los correspondientes túneles y bearers dentro de la arquitectura 5G. Por ello, los

mensajes que aparecen en el log a nivel SDAP permiten identificar qué UE está transmitiendo datos y qué volumen de información se está gestionando.

El campo **ue** permite identificar al usuario, mientras que **pdu_len** indica el tamaño de la PDU transmitida. Sumando estos valores en ventanas temporales de un segundo, es posible obtener una estimación del volumen de datos transferido por cada UE.

3 Simulación de generación de logs: log-sim.py

Antes de procesar los datos, se utiliza el script **log-sim.py**, cuya finalidad es simular la generación progresiva de logs en tiempo real. Este script lee un fichero de entrada:

```
inp_log_file = os.path.join("cu-lan-ho.log")
```

y genera un fichero de salida:

```
out_log_file = "cu-lan-ho-live.log"
```

El fichero **cu-lan-ho.log** contiene el log original capturado durante la práctica. En cambio, **cu-lan-ho-live.log** es el fichero que se va actualizando línea a línea durante la simulación.

El script comprueba primero que el fichero de entrada exista:

```
if not os.path.exists(inp_log_file):
    print("ERROR: File '" + inp_log_file + "' does not exist")
    sys.exit()
```

También verifica que el fichero no esté vacío:

```
if os.path.getsize(inp_log_file) == 0:
    print("ERROR: File '" + inp_log_file + "' is empty (size = 0 bytes)")
    sys.exit()
```

Estas comprobaciones son importantes porque evitan que el programa principal intente procesar datos inexistentes o incompletos.

Una parte fundamental del simulador es la detección de marcas temporales dentro de cada línea del log:

```
def find_dt_pos(text):
    pattern = r'\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{6}'
```

Esta función busca fechas en formato ISO 8601. A partir de ellas, el simulador calcula la diferencia temporal entre eventos del log original y reproduce esa diferencia durante la generación del fichero **cu-lan-ho-live.log**.

La sincronización se realiza mediante este bloque:

```

ellapsed_log = dt_log - dt_ref_log
ellapsed_now = dt_now - dt_ref_now

if ellapsed_now >= ellapsed_log:
    break
else:
    time.sleep(0.01)

```

De esta forma, el script no copia el log de golpe, sino que escribe las líneas respetando aproximadamente la separación temporal original. Esto permite que el programa de monitorización trabaje como si estuviera conectado a una CU real generando logs en directo.

4 Arquitectura general de la solución desarrollada

La solución implementada se divide en cuatro bloques principales:

simulador de logs, productor de líneas, consumidor y parser, agregador y visualización en Dash.

El flujo de funcionamiento es el siguiente:

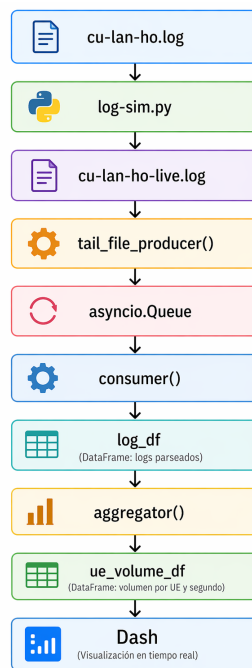


Figure 1: Flujo de funcionamiento del sistema

5 Objetivos

5.1 Leer en tiempo real un log generado por una CU

El primer objetivo consiste en leer en tiempo real el fichero `cu-lan-ho-live.log`, que es generado por `log-sim.py`.

En el programa principal se define el fichero de entrada:

```
LOG_FILE = "cu-lan-ho-live.log"
```

La lectura en tiempo real se realiza mediante la función:

```
async def tail_file_producer(path: str, data_queue: asyncio.Queue) -> None:
```

Esta función espera a que el fichero exista:

```
while not file.exists():  
    await asyncio.sleep(0.5)
```

Esto fue necesario porque, en algunos casos, el programa de monitorización podía ejecutarse antes que el simulador. Si no se añadiera esta espera, aparecería un error al intentar abrir un fichero inexistente.

Una vez creado el fichero, se abre en modo lectura:

```
with open(path, "r", encoding="utf-8") as f:
```

Después se coloca el puntero al final del fichero:

```
f.seek(0, 2)
```

La finalidad de esta línea es procesar únicamente las líneas nuevas que se añadan a partir de ese momento, evitando leer información antigua ya escrita en el fichero.

Cuando aparece una nueva línea, se introduce en una cola asíncrona:

```
await data_queue.put(line)
```

Este diseño permite que la lectura del fichero y el procesamiento de los datos funcionen de manera independiente.

5.2 Identificar los mensajes con información de volumen de datos en DL a nivel SDAP

El segundo objetivo de la práctica consiste en identificar dentro del log aquellos mensajes que contienen información relevante sobre el volumen de datos transferido en el enlace descendente (Downlink), específicamente a nivel de la capa SDAP.

En el contexto de srsRAN, la capa SDAP (Service Data Adaptation Protocol) es responsable de mapear los flujos de datos de usuario hacia los *bearers*

correspondientes, por lo que es en esta capa donde se generan los mensajes que incluyen información sobre el tráfico transmitido.

Previo a extraer los datos definiremos una máscara la cual nos permitirá extraer únicamente los paquetes de datos de tipo SDAP, concretamente los de bajada (CU a UE)

```
LOG_PATTERN = re.compile( #mascara para extraccion de datos lives | solo DL SDAP
    r"^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+).*?\[SDAP\s*\].*?ue=(\d+).*?DL: TX PDU.*?pdu_
```

Esta máscara nos permitirá localizar los paquetes con el siguiente formato (fecha ISO 8001, Etiqueta SDAP, y un dato de tipo pdu_{len}).

A continuación se parsea cada línea del log *live* mediante una expresión regular que permite extraer los campos. Primero se modifica el patrón de búsqueda:

```
m = LOG_PATTERN.match(line)###modificar patron de line
```

Para en el caso de que se haya encontrado una nueva línea en el log *live* que siga la estructura del patrón se le asocia

```
if m:
    ts_str, ue, pdu_len = m.groups()
```

Para verificar el correcto funcionamiento se muestran por consola los parámetros leídos:

```
print("AGG1", f"{ts_str} [{ue}] [{pdu_len}] ") #AGG Verifico los datos leidos
```

Este bloque permite visualizar en la consola únicamente los mensajes provenientes de la capa SDAP, facilitando la comprobación de que el sistema está procesando correctamente la información relevante. Dentro del contenido del mensaje, en particular, se buscan dos campos clave:

- ue: identificador del usuario
- pdu len: tamaño de la unidad de datos (PDU) transmitida

Al Extraer los paquetes de interés a partir de la expresión regular nos permite reducir el coste computacional ya que no tenemos que analizar uno a uno cada paquete. Mas adelante se podría ampliar la expresión regular para localizar paquete con diferente naturaleza, pero por ahora nos quedaremos únicamente con los especificados, De este modo, el sistema descarta automáticamente los mensajes irrelevantes y conserva únicamente aquellos que aportan información sobre el volumen de los datos transferido.

Entre estos campos, el atributo `component` permite identificar el módulo de srsRAN que ha generado el mensaje. En particular, cuando el valor de este campo es `"SDAP"`, se trata de un mensaje relacionado con la capa de adaptación de datos de usuario.

En el código desarrollado, esta identificación se utiliza principalmente con fines de depuración y verificación, mediante el siguiente fragmento:

Este enfoque presenta una ventaja importante: en lugar de depender estrictamente del componente que genera el log, el sistema se centra en la información contenida en el mensaje, lo que lo hace más robusto frente a posibles cambios en el formato de los logs o en la estructura interna de srsRAN.

En conclusión, en este objetivo no se implementa la capa SDAP, sino que se explotan los logs generados por dicha capa para extraer información útil sobre el tráfico de usuario, sentando así la base para el cálculo posterior del volumen de datos por UE.

5.3 Guardar en un DataFrame la información de volumen por UE

Para almacenar los logs procesados se utiliza un DataFrame de Polars:

```
log_df = pl.DataFrame(schema={
    "timestamp": pl.Datetime(time_unit="us"),
    "ue": pl.int_64,
    "pdu_len": pl.int_64,
})
```

Cada línea válida del log se guarda con cuatro campos: `timestamp`, `component`, `level` y `message`.

El consumidor recibe las líneas desde la cola:

```
line = await data_queue.get()
```

Después comprueba si cumplen el patrón definido:

```
m = LOG_PATTERN.match(line)
```

Si la línea es válida, se añade a un buffer temporal:

```
log_buf.append({
    "timestamp": ts,
    "ue": pl.int_64,
    "pdu_len": pl.int_64,
})
```

Para mejorar el rendimiento, no se actualiza el DataFrame línea a línea, sino cada 200 líneas:

```
FLUSH_EVERY = 200
```

Cuando se alcanza ese número, el buffer se concatena con el DataFrame principal:

```
log_df = pl.concat([log_df, pl.DataFrame(log_buf, schema=log_df.schema)], how="vertical")
```

Esta solución evita sobrecargar el programa con demasiadas operaciones de escritura sobre el DataFrame.

5.4 Agregar cada segundo los volúmenes por UE

El cuarto objetivo consiste en calcular el volumen total transmitido por cada UE en ventanas de un segundo.

Para ello se define un segundo DataFrame:

```
ue_volume_df = pl.DataFrame(schema={
    "window": pl.Datetime(time_unit="us"),
    "ue":      pl.Int64,
    "bytes":   pl.Int64,
})
```

Este DataFrame almacena los datos ya agregados.

La agregación se realiza mediante la función:

```
async def aggregator() -> None:
```

Esta función se ejecuta cada segundo:

```
await asyncio.sleep(1)
```

Para no volver a procesar datos antiguos, se usa la variable:

```
new_rows = log_df[last_processed:]
last_processed = len(log_df)
```

A continuación se extraen los campos **ue** y **pdu_len** desde el mensaje del log:

```
pl.col("message").str.extract(r'ue=(\d+)', 1).cast(pl.Int64).alias("ue")
pl.col("message").str.extract(r'pdu_len=(\d+)', 1).cast(pl.Int64).alias("pdu_len")
```

Después se eliminan las líneas que no contienen esos campos:

```
.drop_nulls(["ue", "pdu_len"])
```

Para agrupar por segundos:

```
(pl.col("timestamp").cast(pl.Int64) // 1_000_000 * 1_000_000)
.cast(pl.Datetime(time_unit="us"))
.alias("window")
```

Finalmente se agrupan los datos:

```
.group_by(["window", "ue"])
.agg(pl.col("pdu_len").sum().alias("bytes"))
```

Con esto se obtiene el número total de bytes transmitidos por cada UE en cada segundo.

5.5 Representar en Dash la evolución de los volúmenes agregados

Para la visualización se ha desarrollado una aplicación Dash:

```
app = Dash(__name__)
```

La interfaz contiene un título, una línea de depuración y una gráfica:

```
app.layout = html.Div([
    html.H3("Volume per UE | 5s window"),
    html.Div(id="debug"),
    dcc.Graph(id="vol-graph"),
    dcc.Interval(id="interval", interval=1000, n_intervals=0),
])
```

El componente:

```
dcc.Interval(id="interval", interval=1000, n_intervals=0)
```

actualiza la gráfica cada segundo.

La gráfica se genera en:

```
def update_graph(_):
```

Se seleccionan las últimas ventanas:

```
last_windows = sorted(ue_volume_df["window"].unique().to_list())[-N_SLOTS:]
```

El valor de:

```
N_SLOTS = 5
```

Por tanto, la gráfica muestra una ventana deslizante de 5 segundos.

Para cada UE activo se genera una barra:


```
fig.add_trace(go.Bar(
    x=x,
    y=y,
    name=f"UE {ue_id}",
))
```

Cada UE aparece con un color distinto, facilitando la comparación del volumen de tráfico.

6 Mejoras introducidas en la visualización

Durante las pruebas se observó que algunos UEs podían tener mucho más tráfico que otros. Esto provocaba que las barras de los UEs con menor tráfico apenas fueran visibles.

Para solucionarlo, se añadió una escala logarítmica cuando la diferencia entre el valor máximo y mínimo era muy grande:

```
use_log = b_max > 100 * b_min
```

Si se cumple esta condición, el eje Y cambia a escala logarítmica:

```
yaxis_type="log" if use_log else "linear"
```

Esta mejora permite visualizar simultáneamente UEs con volúmenes de tráfico muy diferentes.

También se fijó el rango del eje Y para evitar saltos bruscos en la gráfica:

```
yaxis_range = (
    [math.log10(max(b_min * 0.5, 1)), math.log10(b_max * 2)]
    if use_log else
    [0, b_max * 1.1]
)
```

Gracias a esto, la representación resulta más estable y fácil de interpretar.

7 Problemas encontrados y soluciones aplicadas

Durante el desarrollo de la práctica surgieron diversos problemas, principalmente relacionados con el procesamiento en tiempo real, la eficiencia en el tratamiento de datos y la gestión de tareas asíncronas. A continuación, se describen los más relevantes junto con las soluciones adoptadas.

7.1 Ineficiencia en la lectura y almacenamiento del log

Inicialmente, el consumidor realizaba una concatenación (`pl.concat`) por cada línea procesada del log. Este enfoque resultaba altamente ineficiente, ya que las operaciones de concatenación sobre DataFrames son costosas cuando se ejecutan de forma repetitiva sobre estructuras pequeñas.

Como consecuencia, el rendimiento del sistema se degradaba significativamente a medida que aumentaba el volumen de datos procesados.

Para solucionar este problema, se implementó un mecanismo de almacenamiento intermedio basado en un buffer de líneas:

```
FLUSH_EVERY = 200
```

Las filas se acumulan en una lista temporal y se insertan en el DataFrame únicamente cuando se alcanza un número determinado de elementos:

```
log_df = pl.concat([log_df, pl.DataFrame(log_buf, schema=log_df.schema)], how="vertical")
```

Este procesamiento por lotes reduce considerablemente el número de operaciones de concatenación, mejorando así el rendimiento global del sistema.

7.2 Inestabilidad visual en la gráfica (eje Y)

Durante la visualización en Dash se observó que el eje Y de la gráfica se reescalaba automáticamente en cada actualización (cada segundo). Esto provocaba un efecto visual de inestabilidad, dificultando la interpretación de los datos.

El origen del problema radicaba en el recalculado dinámico del rango del eje Y en función de los valores visibles en cada instante.

Para solucionarlo, se decidió fijar el rango del eje Y utilizando los valores mínimo y máximo históricos:

```
b_max = all_bytes.max() or 1
b_min = all_bytes.filter(all_bytes > 0).min() or 1
```

Además, cuando la diferencia entre ambos valores supera un determinado umbral, se activa automáticamente una escala logarítmica:

```
use_log = b_max > 100 * b_min
```

Esta solución permite estabilizar la representación gráfica y mejorar la legibilidad cuando existen diferencias significativas de tráfico entre UEs.

7.3 Interrupciones durante operaciones críticas

Otro problema detectado fue la interrupción del programa mediante `Ctrl+C` durante la ejecución de operaciones críticas, como `pl.concat`. Esto podía dejar el sistema en un estado inconsistente o generar excepciones inesperadas.

La causa principal era la ausencia de una gestión adecuada del ciclo de vida de las tareas asíncronas (*producer*, *consumer* y *aggregator*).

Para solucionarlo, las tareas se almacenaron en una lista:

```
tasks = [
    asyncio.create_task(...),
]
```

Y se añadió un bloque de finalización en la función principal para cancelarlas correctamente:

```
finally:
    for t in tasks:
        t.cancel()
```

Esto permite finalizar la ejecución de forma controlada, evitando estados inconsistentes.

7.4 Problemas en el cierre de la aplicación

Durante las pruebas se observó que, en algunos casos, era necesario pulsar varias veces **Ctrl+C** para finalizar completamente el programa. Asimismo, podían aparecer errores relacionados con la finalización de hilos (*threading*), especialmente durante el cierre del servidor Dash.

Este comportamiento se debe a que Dash (basado en Flask) se ejecuta en un hilo separado mediante:

```
asyncio.to_thread(app.run, ...)
```

Al no gestionarse correctamente la terminación de estos hilos, podían producirse bloqueos o excepciones durante el apagado.

Para mitigar este problema se adoptaron las siguientes medidas:

- Desactivación del recargador automático de Flask:
`use_reloader=False`
- Terminación forzada del proceso tras finalizar la ejecución:
`os._exit(0)`

Estas soluciones permiten garantizar un cierre limpio del programa, evitando bloqueos y comportamientos inesperados.

8 Análisis de resultados

En esta sección se presentan algunas capturas representativas del comportamiento del sistema desarrollado durante la ejecución en tiempo real.

8.1 Inicio del sistema sin datos

En esta fase se observa el estado inicial del sistema, donde no se han procesado aún líneas del log (`log rows: 0`) ni se han generado agregaciones (`agg rows: 0`). La gráfica aparece vacía, lo cual confirma que el sistema espera correctamente a la llegada de datos.

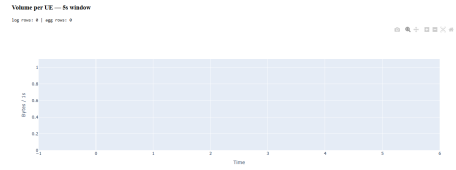


Figure 2: Estado inicial del sistema sin datos procesados

8.2 Primeros datos agregados

En esta fase comienzan a aparecer datos agregados. Se observa la actividad de dos UEs, donde uno presenta un volumen de tráfico superior al otro. La escala logarítmica permite visualizar ambos valores simultáneamente sin perder información.



Figure 3: Primeras agregaciones de tráfico por UE

8.3 Incremento del número de UEs

A medida que avanza la simulación, se incorporan nuevos UEs. Se aprecia cómo el sistema es capaz de representar múltiples usuarios simultáneamente, manteniendo una diferenciación clara mediante colores.

8.4 Comparación de volúmenes heterogéneos

En esta figura se observa una gran diferencia entre los volúmenes de tráfico de distintos UEs. La utilización de escala logarítmica resulta clave para poder representar tanto valores altos como bajos en una misma gráfica.

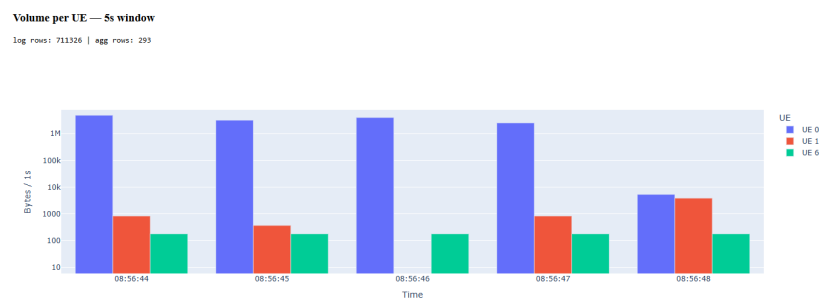


Figure 4: Aparición de nuevos UEs en la red

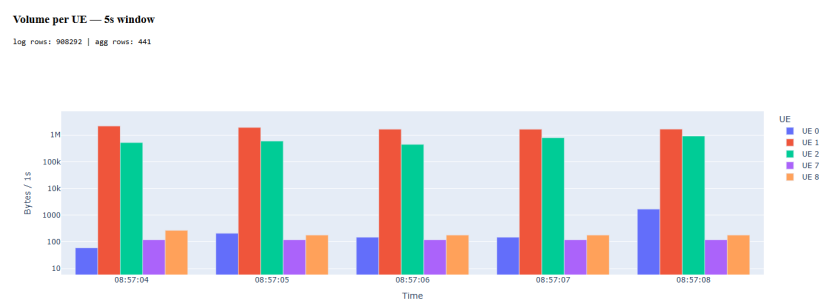


Figure 5: Comparación de tráfico entre múltiples UEs

8.5 Evolución temporal del tráfico

La gráfica muestra la evolución del tráfico en ventanas deslizantes de 5 segundos. Se puede observar cómo el volumen de datos varía dinámicamente en función de la actividad de cada UE.

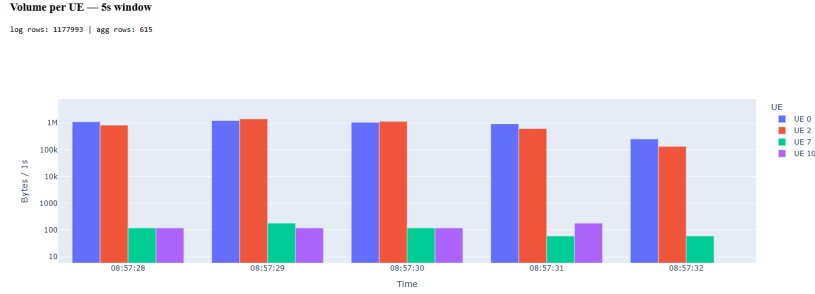


Figure 6: Evolución temporal del tráfico en varios UEs

8.6 Alta densidad de usuarios

En escenarios con un número elevado de usuarios, el sistema sigue siendo capaz de representar la información de manera clara. Cada UE mantiene su color, facilitando el seguimiento individual del tráfico.

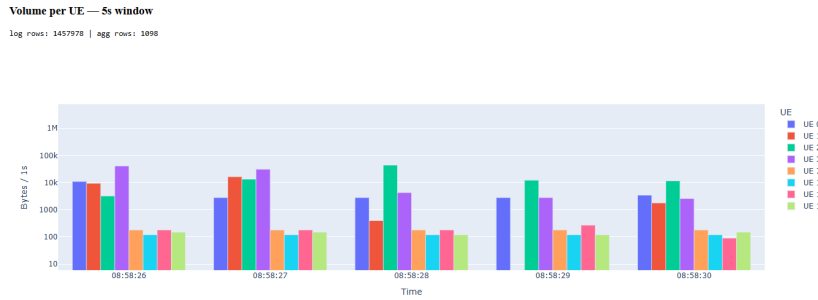


Figure 7: Escenario con múltiples UEs activos

8.7 Estabilidad del sistema

Finalmente, se observa que el sistema mantiene un comportamiento estable a lo largo del tiempo, con actualizaciones continuas y coherentes en la gráfica. Esto valida el correcto funcionamiento de la arquitectura asíncrona implementada.

9 Conclusiones

Este proyecto da respuesta a una pregunta concreta y práctica en entornos de laboratorio 5G: ¿cuántos datos está generando cada UE, cómo evoluciona ese volumen en el tiempo y es posible observarlo en tiempo real? Se trata de una pregunta de observabilidad fundamental, y la solución desarrollada la responde de forma directa: leyendo los logs de la CU tal y como se generan, extrayendo la información de tráfico relevante y representándola en una gráfica actualizada segundo a segundo.

El resultado es una herramienta ligera que no requiere modificar la infraestructura de srsRAN ni instrumentar el código fuente, sino que se apoya exclusivamente en los logs que el sistema ya produce. Esto la hace fácilmente adaptable a otros entornos o versiones de srsRAN con cambios mínimos. Más allá del caso concreto, el proyecto demuestra que un pipeline de telemetría completo y observable puede construirse íntegramente en Python —combinando `asyncio`, `Polars` y `Dash`— sin depender de ninguna infraestructura externa como brokers de mensajes, bases de datos o herramientas de observabilidad de terceros.

Desde el punto de vista técnico, la arquitectura productor-consumidor basada en `asyncio` ha demostrado ser una solución robusta para separar la lectura del log de su procesamiento. La cola asíncrona actúa como buffer entre ambos extremos del pipeline, de forma que si el consumidor se retrasa puntualmente, las líneas pendientes esperan sin perderse. Esto es especialmente relevante en entornos reales, donde los logs no llegan a un ritmo constante sino en ráfagas.

El agregador, por su parte, convive con el hilo del servidor `Dash` sin provocar bloqueos ni retrasos visibles. Al procesar únicamente las filas nuevas en cada ciclo, la latencia desde que una línea aparece en el log hasta que se refleja en la gráfica se mantiene por debajo de un segundo, lo que es adecuado para los objetivos de monitorización planteados.

Finalmente, la expresión regular implementada ha cubierto correctamente el formato de cabecera de la CU de srsRAN durante toda la simulación. No obstante, cualquier cambio en ese formato podría provocar que algunas líneas no se capturasen. Una forma sencilla de detectar este tipo de pérdidas sería comparar el número de líneas del fichero de entrada con el número de filas válidas en el `DataFrame`, o verificar que no existan huecos en las ventanas temporales de la agregación. En cualquier caso, el sistema sienta una base sólida sobre la que incorporar mejoras futuras, como la detección de anomalías en el tráfico o la integración con sistemas de alertas en tiempo real.